Lectura 7: Problemas comunes (overfitting/underfitting) y técnicas de mitigación

SI3015 - Fundamentos de Aprendizaje Automático

2026

1. Overfitting/Underfitting

El overfitting (o sobreajuste) es uno de los conceptos más importantes y, sinceramente, más frustrantes en el Machine Learning.

Imagina que estás estudiando para un examen de historia, pero en lugar de entender por qué ocurrieron las guerras, te memorizas exactamente las palabras de cada página del libro. Si en el examen te preguntan algo con una redacción distinta, fallarás porque no aprendiste el concepto, solo memorizaste los datos. Eso es el overfitting.

En el overfitting, el modelo se vuelve tan flexible y complejo que empieza a "dibujarüna línea que pasa por absolutamente todos los puntos de tus datos de entrenamiento, incluyendo el ruido y los errores aleatorios.

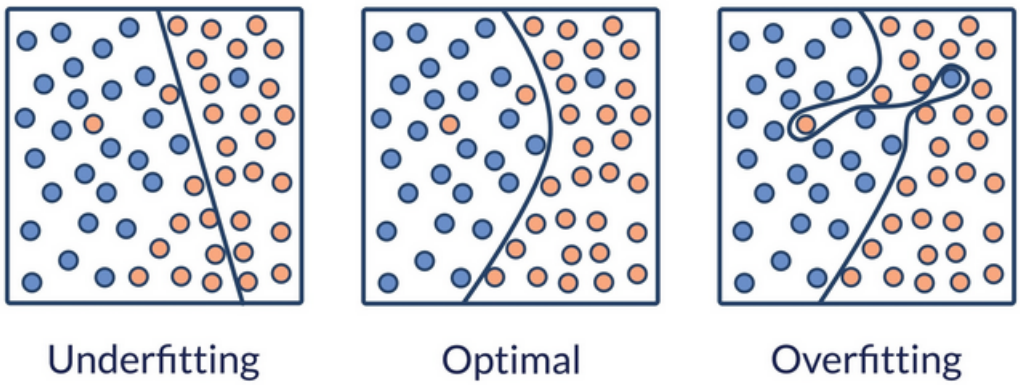

Figura 1: Overfitting y underfitting

Cuadro 1: Comparativa de niveles de ajuste en Machine Learning

Concepto	**Lo que hace el modelo**	**Resultado**
Bajo ajuste (Underfitting)	Es demasiado simple para ver el patrón.	Error alto en entrenamiento y prueba.
Ajuste Óptimo	Captura la tendencia real (la "esencia").	Error bajo en entrenamiento y prueba.
Sobreajuste (Overfitting)	Memoriza los datos y el ruido.	Error bajísimo en entrenamiento, pero muy alto en datos nuevos.

1.1. Causas del Overfitting

1.1.1. Modelo demasiado complejo

Tienes una red neuronal gigante o un árbol de decisión con demasiadas ramas para un problema sencillo. Se presenta la fragmentación del espacio, es decir, el algoritmo sigue creando divisiones (nodos) hasta que cada "hojaçontiene un solo dato o es perfectamente pura. Si tenemos un dato erróneo (un outlier), un árbol complejo creará una rama específica solo para clasificar ese error correctamente. Finalmente, en lugar de aprender que "las casas grandes son caras", el árbol aprende que "la casa de la calle X, con 3 ventanas y una mancha en la puerta, cuesta exactamente 405,321 dólares". Esa regla no le servirá para la casa de al lado.

1.1.2. Pocos datos

El modelo no tiene suficientes ejemplos para generalizar, así que se aferra a lo poco que tiene. En otras palabras, cuando un modelo sufre de escasez de datos, el problema técnico central es que la distribución empírica (lo que el modelo ve en el entrenamiento) no coincide con la distribución teórica real de los datos en el mundo real. En estadística, todo dato tiene dos componentes: **Señal** (el patrón real) y **Ruido** (variaciones aleatorias, errores de medición o factores no controlados). En datasets grandes, el ruido tiende a cancelarse (la media del ruido suele ser cero). El modelo puede "ver" a través de la estática para encontrar la tendencia. En cambio, en datasets pequeños, el ruido no se cancela. Por puro azar, un pequeño grupo de datos puede presentar una correlación espuria.

Por otro lado, a medida que se añaden variables (dimensiones) a tu modelo, el volumen del espacio de búsqueda crece exponencialmente, pero tus datos no. Para que un modelo aprenda una frontera de decisión robusta, necesita ejemplos "cerca" de esa frontera. Con pocos datos, los puntos están aislados en un espacio multidimensional inmenso. Al haber tantos huecos vacíos entre puntos, el modelo "conecta los puntos" de la manera más errática posible para minimizar su pérdida, resultando en una función que no tiene sentido físico o lógico en las zonas donde no hubo ejemplos.

Un dataset pequeño rara vez captura los casos de borde (edge cases) o la varianza total de la población. Por ejemplo, si se entrena un modelo de reconocimiento facial con solo 100 fotos de personas de una misma región, el modelo "se aferrará.ª rasgos específicos

de ese grupo (como el tono de piel o la forma de los ojos) para clasificar la identidad, fallando estrepitosamente al ver a una persona de otra etnia. No es que el modelo sea "malo", es que su "mundo" era demasiado pequeño.

1.1.3. Entrenamiento excesivo

Se permitió que el algoritmo iterara (aprendiera) tantas veces que terminó encontrando patrones donde solo había azar. En otras palabras, el entrenamiento excesivo, técnicamente conocido como **overtraining**, ocurre cuando el proceso de optimización de un modelo se prolonga más allá del punto de máxima generalización. En términos de cálculo, el algoritmo continúa minimizando la función de pérdida empírica a costa de aumentar el error de generalización.

Durante el entrenamiento, un optimizador (como el Descenso de Gradiente) busca el punto más bajo en una superficie matemática compleja llamada *Loss Landscape*, lo cual se puede dividir en dos fases:

- Fase Inicial (Aprendizaje de Estructura): El modelo cae en "valles" anchos y planos que representan patrones generales que se aplican a casi todos los datos (ej. en fotos de perros, aprende que tienen orejas y hocico).
- Fase de Entrenamiento Excesivo: Si no se detiene, el modelo empieza a buscar "agujeros" o mínimos extremadamente estrechos y profundos. Estos mínimos corresponden a configuraciones de pesos que explican perfectamente un dato ruidoso específico, pero que son tan inestables que, si mueves un poco la entrada, el resultado cambia drásticamente.

Técnicamente, el aprendizaje es un proceso de convergencia. Sin embargo, existe un "punto de inflexión" crítico que se describe a continuación:

- Zona de Coherencia: Tanto la curva de error de entrenamiento como la de validación bajan. El modelo está extrayendo información útil.
- Punto de Sobreajuste: La curva de entrenamiento sigue bajando (el modelo está feliz porque su error es casi cero), pero la curva de validación empieza a subir.
- Resultado: El modelo está "alucinando" patrones en el azar. Ha pasado de ser un estudiante que entiende la materia a uno que ha memorizado el orden de las respuestas del examen del año pasado.

En la investigación moderna de aprendizaje automático, se ha descubierto un fenómeno curioso: a veces, si se continua entrenando un modelo mucho más allá del overfitting, el error de validación vuelve a bajar. Esto ocurre en modelos con muchísimos parámetros (como las redes neuronales profundas).

Sin embargo, para la mayoría de los casos prácticos, el entrenamiento excesivo simplemente degrada la utilidad del modelo porque este se vuelve demasiado sensible a la varianza de la muestra de entrenamiento.

2. Técnicas de mitigación

Para mitigar el overfitting (sobreajuste), el objetivo técnico es reducir la varianza del modelo sin aumentar excesivamente el sesgo. Básicamente, queremos obligar al modelo a que deje de memorizar puntos específicos y empiece a aprender la estructura general.

Las principales técnicas de mitigación divididas por su enfoque son:

2.1. Regularización (L1 y L2)

Es la técnica matemática más común. Consiste en añadir un término de penalización a la función de pérdida para evitar que los coeficientes o pesos (θ) crezcan demasiado.

- Regularización L_2 (Ridge): Añade el cuadrado de los pesos a la pérdida ($\lambda \sum \theta^2$). Esto obliga a que los pesos sean pequeños y distribuidos, suavizando la función de decisión.
- Regularización L_1 (Lasso): Añade el valor absoluto de los pesos ($\lambda \sum |\theta|$). Tiene la propiedad de llevar algunos pesos exactamente a cero, lo que funciona como una selección automática de variables (descarta las que no aportan).

2.2. Early Stopping (Parada Temprana)

Es una técnica de optimización temporal. Durante el entrenamiento, monitoreamos el error en un conjunto de datos que el modelo no usa para aprender (set de validación). Detenemos el entrenamiento en el momento exacto en que el error de entrenamiento sigue bajando pero el error de validación comienza a subir. Lo anterior funciona ya que evita que el optimizador entre en la fase donde empieza a ajustar el ruido de los datos de entrenamiento.

2.3. Data Augmentation

Crear datos sintéticos a partir de los existentes (rotar imágenes, añadir ruido blanco a audios, usar sinónimos en texto). A continuación, se describen las estrategias principales por tipo de aplicación:

2.3.1. Visión por Computadora (Imágenes)

Es el campo donde el aumento de datos es más robusto. Se busca que el modelo aprenda características semánticas y no solo posiciones de píxeles.

- **Transformaciones Geométricas:** Rotaciones, traslaciones, espejado (*flipping*) y zoom. Esto enseña al modelo que un "gato" sigue siendo un gato aunque esté de cabeza o en una esquina de la foto.
- **Transformaciones de Color (Photometric):** Ajustar el brillo, contraste, saturación o añadir ruido gaussiano. Esto hace que el modelo sea robusto ante diferentes condiciones de iluminación.

- **Cutout y Mixup:** Técnicas avanzadas donde se eliminan partes de la imagen (*Cutout*) o se mezclan dos imágenes distintas (*Mixup*) con sus etiquetas combinadas.

2.3.2. Procesamiento de Lenguaje Natural (NLP)

Aumentar texto es más complejo porque cambiar una sola palabra puede alterar completamente la semántica de la oración.

- **Reemplazo por Sinónimos:** Utilizar diccionarios como *WordNet* para sustituir términos por otros equivalentes.
- **Retro-traducción (Back-translation):** Traducir una frase a otro idioma y luego de vuelta al original. El resultado suele ser una paráfrasis que conserva el sentido original.
- **Inserción/Eliminación Aleatoria:** Añadir o quitar palabras no críticas para que el modelo identifique las *keywords* reales.

2.3.3. Procesamiento de Audio

En audio, las variaciones físicas son fundamentales para el entrenamiento de modelos de reconocimiento de voz.

- **Inyección de Ruido:** Añadir ruido blanco o de fondo (lluvia, tráfico). Es vital para que el modelo funcione en entornos reales.
- **Time Stretching:** Acelerar o ralentizar el audio sin cambiar el tono (*pitch*). Ayuda a reconocer palabras independientemente de la velocidad del habla.
- **Pitch Shifting:** Cambiar el tono de la voz para simular diferentes timbres y generalizar entre distintos hablantes.

2.3.4. ¿Por qué éstas técnicas funcionan contra el Overfitting?

Desde la perspectiva de la optimización, el *Data Augmentation* actúa como una forma de **regularización**:

1. **Suavizado de la vecindad:** Al crear variantes, se le indica al modelo que toda una región alrededor de ese punto pertenece a la misma categoría, suavizando la frontera de decisión.
2. **Reducción de la memorización:** Como el modelo rara vez ve la misma entrada exacta, se ve obligado a aprender patrones generales en lugar de memorizar valores específicos.

2.4. Transfer Learning

Transfer Learning consiste en tomar un modelo que ya es "experto" en un tema y usar su conocimiento previo para que aprenda una tarea nueva y parecida, ahorrando tiempo y esfuerzo.

Aunque en los modelos clásicos (como Random Forests o SVM) no hay "neuronas"que copiar, existen formas ingeniosas de pasar el conocimiento de un modelo a otro:

2.4.1. Transferencia por Re-pesado (Instancias)

Si se tienen muchos datos de un problema viejo y pocos del nuevo, puedes mezclarlos, pero dándole más importancia a los datos viejos que más se parecen a los nuevos. El algoritmo analiza los datos del "problema Az les asigna un peso. Si un dato del pasado contradice lo que está pasando en el "problema B", el modelo lo ignora. Si lo apoya, lo usa para fortalecer su aprendizaje. Es ideal para Regresión Logística y SVM.

2.4.2. Transferencia de Características (Variables)

A veces, lo más valioso no son los datos en sí, sino saber qué variables son las que importan. Se entrena un modelo clásico (como un Random Forest) en un dataset gigante. Se identifica cuáles son las 5 variables que realmente definen el resultado. Luego, construimos el modelo nuevo usando solo esas 5 variables seleccionadas. Es ideal para Árboles de Decisión y Gradient Boosting.

2.4.3. Transferencia de Reglas (Relacional)

Consiste en llevarse la "lógica" o las reglas de un negocio de un lugar a otro. Si un árbol de decisión aprendió en España que "si el cliente tiene más de 60 años, es propenso a comprar el producto X", se podría usar esa misma regla como punto de partida (o como una variable extra) para un modelo que se está creando en México. No se copian los datos, se copia la conclusión.

2.4.4. Adaptación del Espacio (Traducción)

Si los datos del problema A y del problema B hablan "idiomas" distintos, se intenta buscar un punto medio. Aquí se usa una técnica matemática (como PCA) para proyectar ambos grupos de datos en un nuevo gráfico donde se vean parecidos. Una vez que están en el mismo "formato", el modelo clásico puede aprender de ambos simultáneamente.

2.5. Validación cruzada (Cross-Validation)

La Cross-Validation (Validación Cruzada) es una técnica estadística utilizada para evaluar la capacidad de generalización de un modelo. En lugar de confiar en una única división de los datos, este método fragmenta el dataset repetidamente para asegurar que el modelo funcione bien con diferentes subconjuntos de información. La variante más común es el K-Fold Cross-Validation.

Se dividen los datos en K grupos iguales (llamados folds). Por ejemplo, si K=5, entonces tenemos 5 grupos. El proceso se repite 5 veces. En cada ronda se toma un grupo diferente como "Set de Prueba" (o Validación). Los otros 4 grupos se usan como "Set de Entrenamiento". Al final, se calcula el promedio de los errores de las 5 rondas. Ese número es la verdadera medida de qué tan bueno es tu modelo.

El overfitting ocurre cuando un modelo "se memoriza" el ruido de un dataset específico. La Cross-Validation combate esto por tres razones principales:

- Evita el "Sesgo de Selección": Si se realiza una sola división (ej. 80 % entrenamiento, 20 % prueba), podrías tener la mala suerte de que el 20 % de prueba sea "muy fácil"

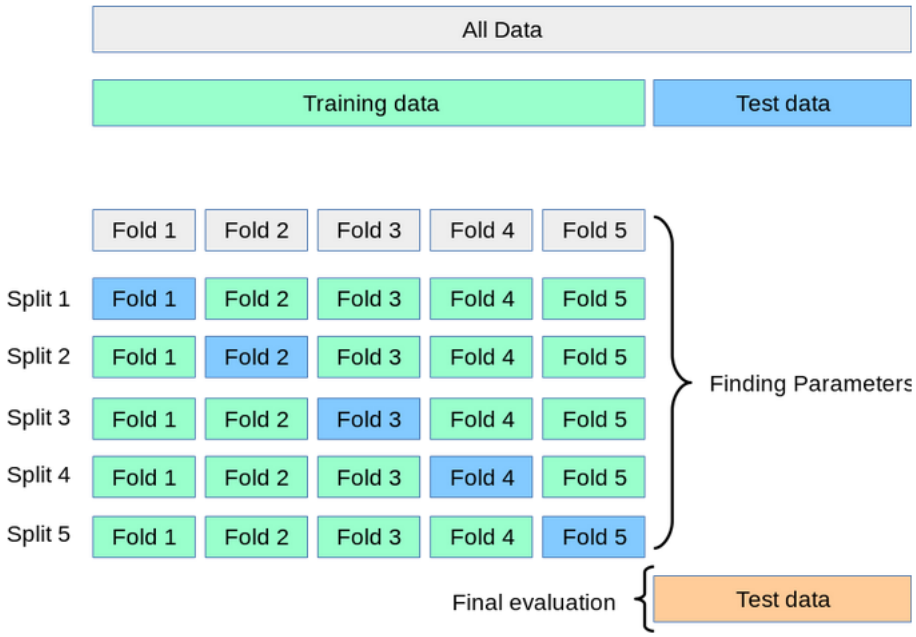

Figura 2: Vaidación cruzada

o "muy difícil" de predecir por puro azar. Esto te daría una falsa sensación de éxito. La Cross-Validation obliga al modelo a probarse contra todos los datos en algún momento, eliminando la suerte del camino.

- Detección de Inestabilidad (Varianza Alta): Si al aplicar Cross-Validation se observa que en el Fold 1 el modelo tiene una precisión del 95 %, pero en el Fold 3 cae al 60 %, has detectado overfitting. Una desviación estándar alta entre los resultados de los folds significa que el modelo es muy sensible a pequeños cambios en los datos (clásica señal de sobreajuste).

- Optimización de Hiperparámetros: Es la herramienta reina para decidir qué tan complejo debe ser un modelo. Preguntas como: ¿Cuántas ramas debe tener mi árbol? o ¿Qué tan fuerte debe ser mi regularización λ? pueden ser halladas al probar diferentes configuraciones mediante Cross-Validation. Se elige la que da el mejor resultado promedio, no la que mejor se ajusta a un solo grupo de datos. Esto garantiza que la configuración elegida sea robusta.